

CineVerse Implementation Plan v2.0

1. Goal

Finish the complete basic movie-ticket booking lifecycle first, then layer advanced integrations. The supplied baseline proposed six sprints. This plan keeps the dependency order but changes sequencing so the team reaches a working end-to-end demo earlier: real payment and AI no longer block the middle of the project. [Source: provided CineVerse specification]

Priority policy for the current build

Priority	Meaning	Delivery rule
P0 - NOW / MUST SHIP	Core product must work end-to-end without these features the project is incomplete.	Implement before moving to polish.
P1 - NEXT / IMPORTANT	Strong product completeness, but does not block the first working ticket lifecycle.	Build immediately after P0.
P2 - DEFERRED	Useful production capability, intentionally not on the current critical path.	Design interfaces now; integrate later.
P3 - FUTURE / ADVANCED	AI, wallet, and other advanced experiences.	Do not block the base application.

Explicit project decisions

1. **AI movie recommendation is deferred to P3.** Keep preference fields and event data so the feature can be added later, but do not build the recommendation model now.
2. **Conversational AI booking assistant is deferred to P3.** No LLM dependency is required for the core release.
3. **Real payment gateway integration is deferred to P2.** Build a payment abstraction and a deterministic **Mock Payment Adapter** in P0 so that booking, QR generation, cancellation rules, and scanner flows can be tested end-to-end.
4. **Wallet integrations (Google/Apple or similar) are deferred to P3.** The first release should support a downloadable digital ticket / QR and a simple share action instead.
5. **Smart seat recommendation is not a blocker.** Normal seat selection and real-time locking are P0; the heuristic “Best View / Couple / Cheapest / Near Exit / Accessible” helper is P1/P2 depending on team capacity.

6. **Refund gateway automation is deferred with real payments.** The database must still record cancellation and refund intent so the workflow can be integrated later without changing the booking model.

2. Delivery roadmap

```
PHASE 0  Foundation & repository
|
v
PHASE 1  Auth + RBAC + Admin movie catalogue + Theatre onboarding
|
v
PHASE 2  Screens + Seats + Shows + Pricing
|
v
PHASE 3  Customer discovery + Seat map + Redis locks + WebSocket
|
v
PHASE 4  Checkout + Mock payment + Booking + QR ticket
|
v
PHASE 5  Staff scanner + Manager analytics + Cancellation + Audit
|
v
PHASE 6  P1 polish and production hardening
|
+----> P2 Real payment/refund integration
+----> P3 AI recommendation / AI assistant / wallet
```

3. Phase 0 - Foundation

Backend tasks

- ☐ Create Node.js + Express + TypeScript backend.
- ☐ Add environment/config loader.
- ☐ Add PostgreSQL connection and migration runner.
- ☐ Add Redis connection.
- ☐ Add global error middleware.
- ☐ Add request validation middleware.
- ☐ Add requestId and structured logging.
- ☐ Add OpenAPI/Swagger generation later (P1).

Frontend tasks

- ☐ Create React + Vite + TypeScript app.
- ☐ Tailwind setup.
- ☐ TanStack Query setup.

- ☐ Router with customer/manager/staff/admin route groups.
- ☐ Shared layout, toast, modal, table, form and loading components.

DevOps tasks

- ☐ `.env.example`.
- ☐ Docker Compose for Postgres + Redis.
- ☐ Seed script.
- ☐ Git branching convention and PR rules.

4. Phase 1 - Authentication, RBAC, master data

P0 backend

- ☐ `users`, `roles_permissions`, `refresh_tokens` migrations.
- ☐ Register/login/logout/refresh.
- ☐ Permission middleware.
- ☐ Tenant scope middleware.
- ☐ Account lockout counters.
- ☐ Super Admin movie CRUD.
- ☐ Theatre application state machine.
- ☐ Theatre document metadata.

P0 frontend

- ☐ Customer login/register.
- ☐ Manager/staff/admin login.
- ☐ Admin dashboard shell.
- ☐ Movie CRUD screens.
- ☐ Theatre approval queue.

5. Phase 2 - Theatre setup, seat layout, show scheduling

- ☐ Screen CRUD.
- ☐ Seat template/bulk generation.
- ☐ Seat tier editor.
- ☐ Accessibility and broken-seat flags.
- ☐ Screen preview.
- ☐ Show create/update/cancel.
- ☐ Overlap protection.
- ☐ Base tier pricing.
- ☐ Manager show calendar.

Definition of done

A manager can create a theatre, add a screen, generate 100+ seats, assign tiers, select a catalogue movie, schedule a show, and expose that show to the customer

discovery API.

6. Phase 3 - Customer discovery and real-time seats

- ☐ Home API.
- ☐ Search API.
- ☐ Movie details.
- ☐ Theatre/show matrix.
- ☐ Seat map API.
- ☐ Redis SET NX EX 300 seat locking.
- ☐ Lock ownership check for release.
- ☐ Socket.io show-room events.
- ☐ Lock countdown.
- ☐ 409 seat conflict handling.
- ☐ Concurrency tests.

The supplied architecture explicitly requires Redis atomic locking plus Web-Socket propagation of seat state. [Source: provided CineVerse specification]

7. Phase 4 - Checkout, mock payment, booking, QR

P0

- ☐ Checkout calculation service.
- ☐ Coupon validation.
- ☐ Booking initiation.
- ☐ Mock payment adapter.
- ☐ Idempotency key handling.
- ☐ Final PostgreSQL commit transaction.
- ☐ Seat lock cleanup.
- ☐ QR payload/signature generation.
- ☐ Ticket page.
- ☐ Downloadable PDF ticket.
- ☐ My Bookings.

P1 after basic success

- ☐ F&B cart.
- ☐ Advanced coupon rules.
- ☐ Email confirmation.
- ☐ Push/SMS notifications.

8. Phase 5 - Gate operations, cancellations, analytics

- ☐ Staff roster API.
- ☐ QR camera scanner.
- ☐ Six-point validation.

- ☐ Atomic ticket USED transition.
- ☐ Booking lookup by booking ID.
- ☐ Cancellation policy service.
- ☐ Refund intent record.
- ☐ Manager occupancy dashboard.
- ☐ Daily revenue dashboard.
- ☐ Audit log UI.

The source staff flow requires six checks before entry and passback protection.
[Source: provided CineVerse specification]

9. Phase 6 - P1 polish and hardening

- ☐ Reviews.
- ☐ Watchlist.
- ☐ Theatre offers.
- ☐ F&B.
- ☐ Report exports.
- ☐ Smart seat finder.
- ☐ Dynamic pricing rules.
- ☐ Better analytics.
- ☐ Security audit.
- ☐ Load tests.
- ☐ Backup/restore test.

10. P2 - Real payment integration

Do only after P0 checkout is stable.

Integration checklist

- ☐ Choose gateway.
- ☐ Implement provider adapter behind `PaymentGateway` interface.
- ☐ Create gateway order.
- ☐ Redirect/SDK checkout.
- ☐ Verify callback signature.
- ☐ Persist gateway order/transaction IDs.
- ☐ Handle SUCCESS/FAILED/CANCELLED/EXPIRED.
- ☐ Webhook idempotency.
- ☐ Reconciliation worker.
- ☐ Refund API.
- ☐ Payment monitoring.

The supplied TRD defines webhook idempotency and reconciliation for payment failures; this phase preserves that design. [Source: provided CineVerse specification]

11. P3 - AI recommendation and AI assistant

Recommendation engine

Capture events now but do not model now:

- movie viewed
- show viewed
- booking completed
- movie rated
- genre preference
- language preference
- favourite actor/director

Later, build candidate generation -> ranking -> explanation.

AI assistant

Later route: natural language -> structured search filter -> show selection -> seat selection -> deep link. Keep all booking execution server-side; the AI should never directly mutate inventory.

12. P3 - Wallet

Wallet integration is independent of core QR ticketing. Keep the ticket payload stable so a Google/Apple wallet adapter can later reuse the same booking/ticket identity.

13. Suggested workstreams

Workstream	Main output
Auth / security	JWT, refresh, RBAC, tenant scope, audit
Catalogue / admin	Movies, theatres, approvals, coupons
Theatre engine	Screens, seats, shows, pricing
Customer booking	Discovery, seat map, checkout, bookings
Realtime / backend	Redis locks, Socket.io, transactions
Gate / analytics	QR scan, validation, dashboards

14. Definition of Done for P0

A feature is done only when it has:

- UI + API + database implementation.
- Permission enforcement.
- Validation/error handling.
- At least one automated test.
- Seed/sample data where necessary.

- Loading/empty/error UI states.
- Audit entry for sensitive admin/manager mutations.
- No cross-tenant leakage.

15. Risk register

Risk	Level	Mitigation
Double booking	Critical	Redis lock + PostgreSQL FOR UPDATE
Cross-tenant leak	Critical	Permission claims + theatre scope middleware
Duplicate payment	High	Idempotency keys, but P0 uses mock adapter
Browser closes during hold	Medium	Redis TTL
QR passback	High	Atomic UNUSED -> USED
Overbuilt AI before core	High	Explicit P3 gate
Payment integration delays	High	Mock adapter + provider interface
Large schema changes later	Medium	Stable booking/ticket/payments interfaces from P0